

TREE-01

Q6)

Question:

DS	Session	Tree 1	Question Information	Level 1	Challenge 66
Question description A company has n employees, who form a tree hierarchy where each employee has a boss, except for the general director. Your task is to process q queries of the form: who is employee x's boss k levels higher up in the hierarchy? Input The first input line has two integers n and q: the number of employees and queries. The employees are numbered $1, 2, \dots, n$, and employee 1 is the general director. The next line has $n-1$ integers $e_2, e_3, \dots, e_n$: for each employee $2, 3, \dots, n$ their boss. Finally, there are q lines describing the queries. Each line has two integers x and k: who is employee x's boss k levels higher up? Output Print the answer for each query. If such a boss does not exist, print -1. Constraints • $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq e_i \leq i-1$ • $1 \leq x \leq n$ • $1 \leq k \leq n$					

PROGRAM:

```
1  #include <stdio.h>
2
3  int tree[800005];
4
5  int min(int a, int b) {
6      return a < b ? a : b;
7  }
8
9  void build(int a[], int node, int l, int r) {
10     if (l == r) {
11         tree[node] = a[l];
12         return;
13     }
14
15     int mid = (l + r) / 2;
16
17     build(a, node * 2, l, mid);
18     build(a, node * 2 + 1, mid + 1, r);
19
20     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
21 }
22
23 int query(int node, int l, int r, int a, int b) {
24     if (a <= l && r <= b)
25         return tree[node];
26
27     int mid = (l + r) / 2;
28
29     if (b <= mid)
30         return query(node * 2, l, mid, a, b);
31
32     if (a > mid)
33         return query(node * 2 + 1, mid + 1, r, a, b);
34
35     return min(query(node * 2, l, mid, a, b),
36               query(node * 2 + 1, mid + 1, r, a, b));
37 }
38
39 int main() {
40     int n, q, a[200005];
41
42     scanf("%d %d", &n, &q);
43
44     for (int i = 1; i <= n; i++)
45         scanf("%d", &a[i]);
46
47     build(a, 1, 1, n);
48
49     while (q--) {
50         int l, r;
51         scanf("%d %d", &l, &r);
52         printf("%d\n", query(1, 1, n, l, r));
53     }
54
55     return 0;
56 }
```

OUTPUT:

TEST CASE 1:

```
8 4
3 2 4 5 1 1 5 3
2 4
2
5 6
1
1 8
1
3 3
4
```

TEST CASE 2:

```
10 4
13 12 14 51 11 1 5 3 4 6
3 4
14
3 6
1
4 8
1
1 3
12
85 8 14 1 13 1 13 8 1 13
```